

申请名称

《一种基于知识引导深度强化学习的可重构车间动态调度方法》

摘要

本申请涉及一种基于知识引导深度强化学习的可重构车间动态调度方法。更具体地，本申请设计一种在云边协同架构下，通过知识引导深度强化学习（KGDL）优化可重构制造系统（RMS）动态资源调度与任务分配的实现方法。该方法针对个性化定制生产（CPP）场景中存在的动态重配置时间干扰、高维复杂调度决策以及资源分配不足等问题，结合云边协同架构的实时数据处理能力和无线智慧工厂的柔性重组特性，构建动态调度模型与知识引导网络，实现任务加工次序、机器功能切换及资源重配置的全局优化，从而最小化总完工时间并提升调度效率与稳定性。其核心技术包括：基于马尔可夫决策过程的状态-动作建模、知识引导网络对调度策略的启发式增强、动态重配置时间约束的集成建模，以及云平台与边缘节点的协同决策机制，有效解决了传统柔性制造系统（FMS）在动态场景下调度响应滞后和资源利用率低的问题。

技术领域

本申请涉及一种基于知识引导深度强化学习的可重构车间动态调度方法。更具体地，本申请设计一种在云边协同架构下，通过知识引导深度强化学习（KGDL）优化可重构制造系统（RMS）动态资源调度与任务分配的实现方法。

背景技术

近年来，随着全球制造业技术的不断发展，个性化定制生产（Customized and Personalized Production, CPP）逐渐成为制造业发展的主流趋势。其强调生产过程的灵活性和快速响应能力，旨在实现产品“按需设计”“按需制造”，从而实现高效生产并满足用户多样化需求。为了实现上述目标，现有制造商多选择引入柔性制造系统（Flexible Manufacturing System, FMS）与云边协同架构。FMS 能够显著提升生产效率并灵活适应市场需求，而云边协同架构是面向无线智慧工厂设计

的新型架构，包含云计算平台、制造节点等要素，具有实时数据处理能力，能够根据加工需求重新配置生产设备，从而实现个性化定制生产的目标。然而，柔性制造系统在 CPP 中存在资源分配和动态操作调度不足的问题，在实际应用中仍然面临较大挑战，而当前的云制造生产系统亦由于机器有限连接限制，在个性化生产当中难以承担复杂的操作选择流程，无法灵活地重新配置生产资源。与柔性制造系统相比，可重构制造系统（Reconfigurable Manufacturing System, RMS）因具备允许制造资源进行重新分解配置、能够执行多种工艺作业、提高作业调度灵活性等优势，逐渐在 CPP 流程中占据重要位置。

现有关于柔性制造系统的研究集中于柔性作业车间调度（CN114912826B）、电力经济调度（CN116468106A）和推荐系统（CN117473160A）等领域，使用了图神经网络（GNN）、Actor-Critic 框架、知识增强强化学习等方法。而其相似特点是通过分层决策或知识引导方式缩小动作搜索空间以提升收敛性，在一定程度上提升了设备调度效率，但在动态环境适应性方面存在较多缺陷，尤其无法解决实时重配置时间优化与高维动作空间的协同决策问题，在跨领域知识迁移、云边协同架构设计方面还有较大提升空间。

申请内容

本申请的目的在于提供一种在云边协同架构下基于知识引导深度强化学习的可重构车间动态调度方法，通过部署通用启发式知识引导网络框架加快智能体快速策略迭代，提高车间调度问题的求解速率，从而提高个性化定制生产效率。

一种云边协同架构下基于知识引导深度强化学习的可重构车间动态调度方法，该方法包括以下步骤：

定义个性化定制任务，确定可重构调度车间中 CPP 任务的数量及任务操作、序列、加工阶段类型与工作量；

定义可重配置机器，确定可重构调度车间中可重构机器数量及对应工作状态；

定义资源重配置过程，构建机器与任务的对应关系并确定任务实际加工时间，获得智能体决策依据与所有任务的完工时间。

进一步地，构建：

运用非线性规划模型将调度过程、完工时间和动态重构因素等重配置过程进

行建模，以最小化所有任务的总完工时间。

进一步地，构建基于知识引导的深度强化学习算法 (Knowledge-Guided Deep Reinforcement Learning, KGDRL)，包括：

构建马尔科夫决策过程，定义车间状态向量、任务机器动作向量、状态迁移向量、考虑加工时间的奖励函数和策略向量（需放 MDP 各状态参数）；

建立启发式知识引导网络（放可用动作知识引导的概率分布 4-14），包括：

构建动作概率向量输出函数，加速智能体快速策略迭代流程，增强训练结果的有效性并加快训练速度；

可选地，对于可用机器进行分配时，使用先验知识进行引导；

可选地，在取得动作概率分布时，使用范数为 1 的激活函数；

进一步地，使用近端优化策略 (Proximal Policy Optimization, PPO) 结构进行训练，部署动作-评论网络 (Actor-Critic Network) 进行策略评估迭代，包括：

运用逐步优化策略避免策略震荡或训练不稳定；

使用增强算法确保高效迭代加工任务；

使用包含一层隐藏层的两层深度 Q 网络设计动作-评论网络；

可选地，在 5*5 算例上根据最优调度调整超参数；

可选地，设置迭代次数和实例批量大小分别为 1000 和 10，设置算法最小和最大重放缓冲区域为 128 和 2048；

可选地，设置 PRO 损失裁剪比和熵系数分别为 0.1 和 0.01；

可选地，设置损失迭代次数为 15；

计算 PRO 损失。

进一步地，构建动态重构调度算例用于算法训练，确保 KGDRL 方法在所有 CPP 中的有效性，包括：

可选地，在六个维度上通过正态分布随机生成不同加工时间、重配置时间和处理速度的调度算例；

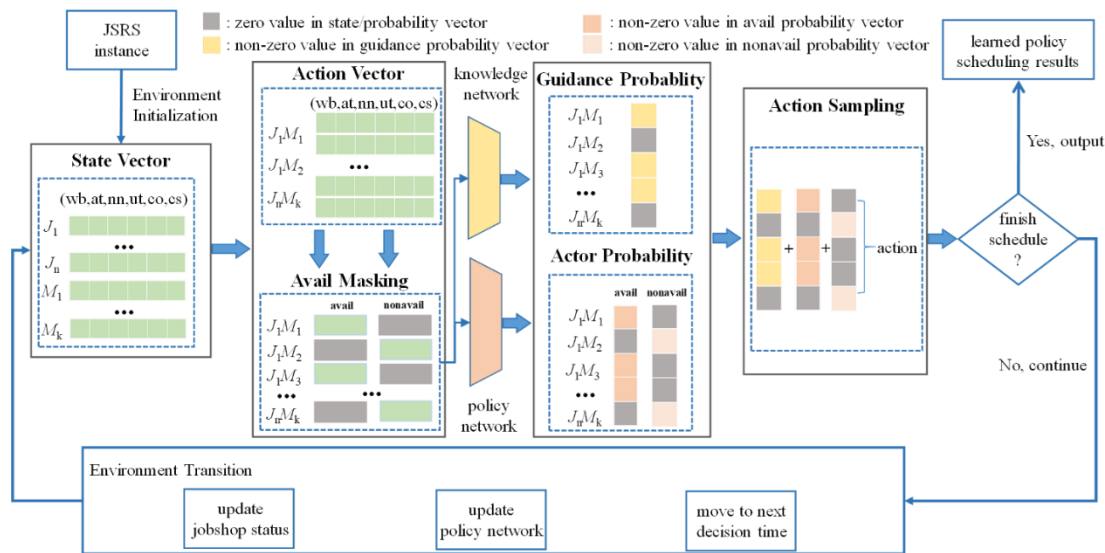
可选地，将平均重新配置时间设置为平均任务阶段加工时间的 0.1 倍，将平均机器处理速度设置为 1。

附图说明

为了更清楚地说明本申请实施例中的技术方案，下面将对实施例描述中所需

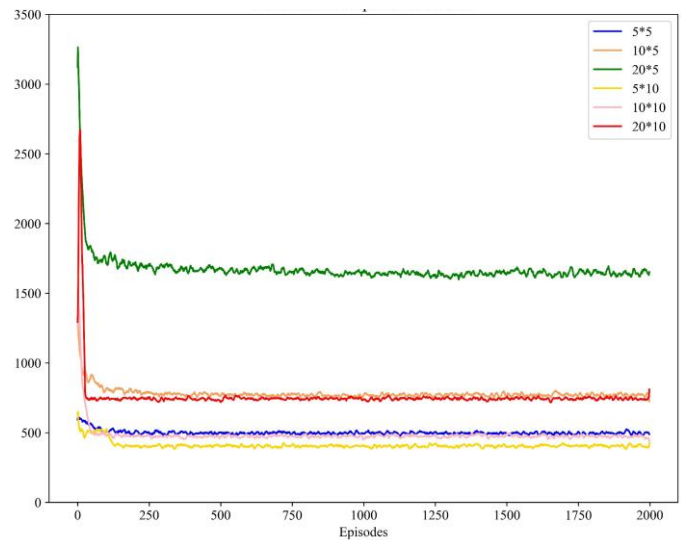
要使用的附图作简要介绍，显而易见地，下面描述中的附图仅仅是本申请的一些实施例，对于本领域的普通技术人员来讲，在不付出创造性劳动的前提下，还可以根据这些附图获得其他的附图。

图 1 是基于知识引导的深度强化学习算法具体运行机制示意



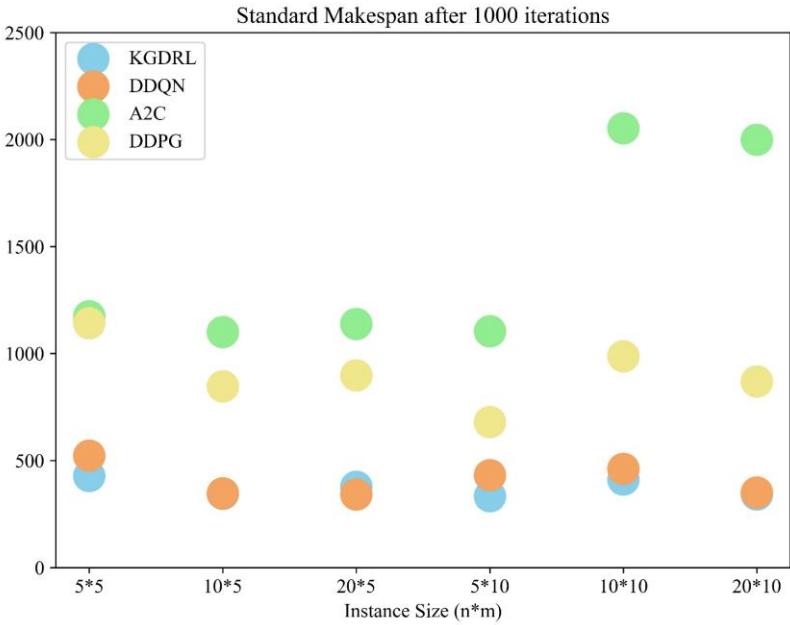
图表 1 基于知识引导的深度强化学习算法具体运行机制示意

图 2 是 KGDL 算法在不同调度算例大小下的训练曲线：



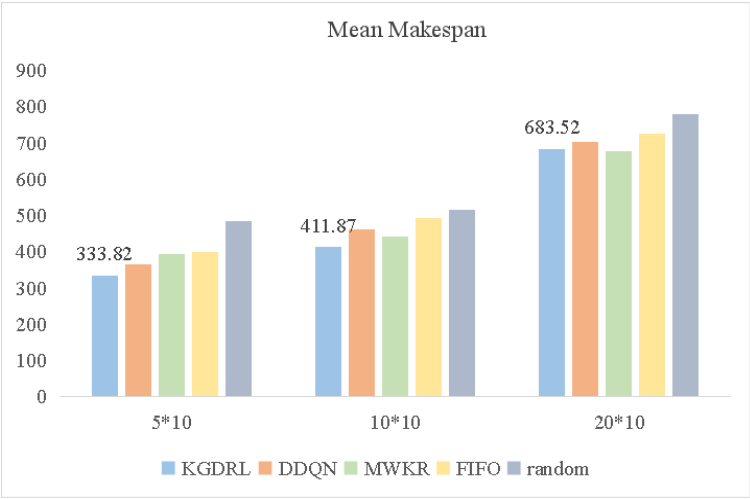
图表 2 KGDL 算法在不同调度算例下的训练曲线

图 3 是 KGDRL 算法与 DDQN 算法和其他三个基线算法的在不同算例下的收敛性结果比较：



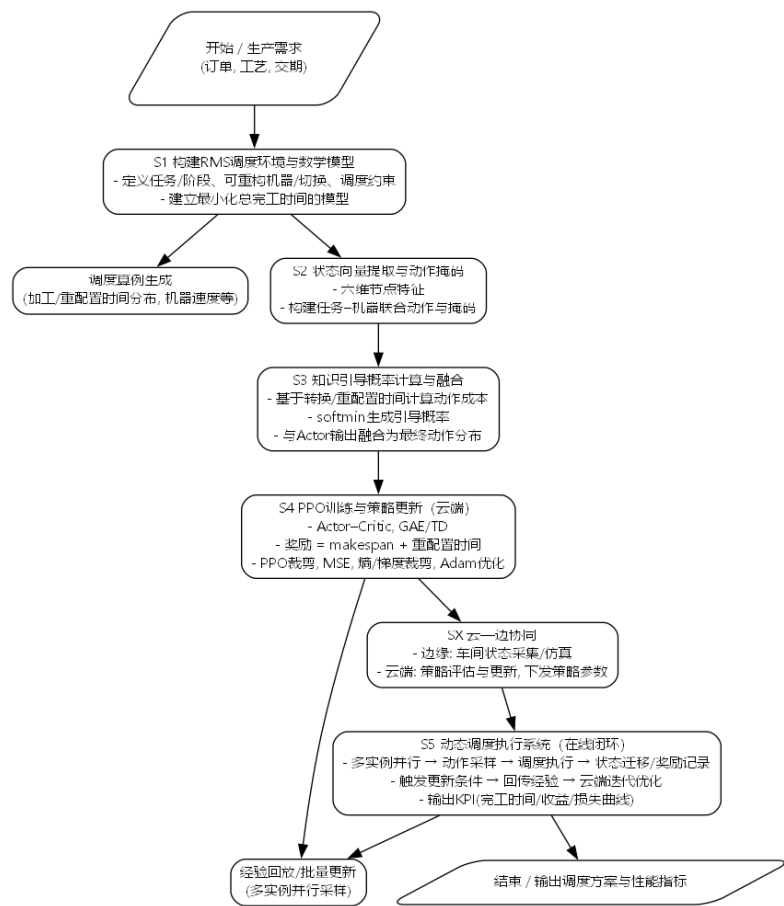
图表 3 KGDRL 算法与基线算法收敛性结果比较

图 4 是 KGDRL 算法在不同算例维度下的调度表现比较：



图表 4 机器数为 10 时 KGDRL 算法算例调度结果

图 5 是 KGDRL 算法的实现流程示意图：



图表 5 KGDRL 算法实现流程示意图

具体实施方式

为使本申请的目的、技术方案以及优点更加清楚，下面将结合云边协同架构下可重构车间动态调度的实例对本申请进行具体说明。显然，所描述的实施例仅仅是本申请一部分实施例，而不是全部的实施例。基于本申请中的实施例，本领域人员在没有做出创造性劳动前提下所获得的所有其他实施例，都属于本申请保护的范围。

如图 1 所示，为本申请实施例提供的基于知识引导深度强化学习的可重构车间动态调度方法示意图。本申请提供的方法包括以下步骤：

步骤 S1，基于可重构机器调度环境进行数学建模

在本申请实施例中，构建可重构调度环境的步骤包括构建个性化定制任务相关变量、可重配置机器相关变量和任务调度过程相关变量。

定义可重构调度车间任务相关变量。

具体地，明确个性化定制生产任务的唯一标识符，明确各生产任务需要经过的阶段列表，记录当前任务所属的阶段、状态及平均处理时间。

定义可重配置机器相关变量。

具体地，明确调度车间内各机器的唯一标识符，明确机器可以处理的任务阶段列表和可用时间节点，设置机器在不同阶段的处理速度与阶段之间的重配置时间，记录可重配置机器处于的任务阶段、预计完成时间和闲置时间。

定义任务调度过程相关变量。

具体地，明确可用于特定操作的机器、任务和阶段组合，实现任务、阶段之间的切换，并检查所有任务是否已经被完成，目的是为强化学习系统提供仿真平台，监控调度状态。

步骤 S2，定义状态与动作向量

在本申请实施例中，定义状态与动作向量步骤包括从明确状态向量构成，从可重构调度环境中提取状态向量特征；明确动作向量构成，构建动作掩码，获取策略网络输入参数。

具体地，定义 t 时刻的状态向量为 $S_t = \{N_t(i), i \in \{J, M\}\}$ ，用于表征车间系统并引导调度决策。其包括所有机器和任务节点的状态向量。每个节点则包含六个特征，令特征函数为 $N_t(i) = (wb_t, at_t, nn_t, ut_t, co_t, cs_t)_i, \forall i \in \{J, M\}$ 。每个特征的具体解释如下：

特征名	释义
wb	节点的工作状态(0 记为空闲，1 记为工作)
at	节点的空闲时间
nn	可以到达的节点数量
ut	节点的时间利用率(工作时间/总时长)
co	当前节点的阶段数
cs	当前节点的加工速度

基于以上特征，通过任务特征和机器特征从环境中提取各作业与机器的可用时间、当前阶段、邻接任务数等关键特征，而后对任务动作对进行特征拼接，形

成状态向量集合。最后用统一标准生成维度一致的状态表示，用于强化学习模型中的策略网络输入。

具体地，定义 t 时刻的动作向量为 $a_t = \{(o_{ij}, M_k)\} \cup \{None\}$ ，表示任务动作对，用于描述可重构生产系统阶段选择与机器分配的联合动作。其中 O_{ij} 代表可加工任务阶段，而 M_k 代表空闲且可加工的机器。None 表示当前状态智能体不采取任何加工动作。动作向量由状态向量聚合而成，即 $S'_t = \text{concat}(N_t(J), N_t(M))$ 。通过可行性判断生成策略层动作过滤机制，构建系列动作掩码。然后通过 sklearn 工具实现向量计算与概率归一化，解决动作空间与非法采样问题，避免不可行动作对策略网络产生干扰。

步骤 S3，构建知识引导网络

构建知识引导网络，生成引导概率分布输出函数，从而加快策略收敛速度，提高策略网络对调度环境的适应能力。

具体地，设计知识引导模块，其依据当前调度状态中机器转换设置时间推导任务优先级，计算动作成本并引导策略网络选择低代价转移路径。该模块输出指导概率，并与策略网络输出进行融合以生成最终动作采样分布，用于引导策略网络输出更具方向性的动作选择。定义可用动作的知识引导概率分布为：

$$S_1 = \text{trantime}(a_t) / \|\text{trantime}(a_t)\|_2, S_2 = \text{workload}(a_t) / \|\text{workload}(a_t)\|_2 \\ p_{a_t} = \alpha_1(e^{-S_1} / \|e^{-S_1}\|_1) + \alpha_2(e^{-S_2} / \|e^{-S_2}\|_1)$$

具体地，提取可行动作集合，提取当前状态下所有可调度的机器与作业对；构造动作空间节点，包括所有可能的机器作业对，用于后续生成引导概率分布；构造可行动作掩码，标记可执行动作；计算转移时间向量，作为动作成本衡量指标；使用 softmax-like 方法生成引导概率分布，输出概率向量，作为机器作业动作对引导概率。

步骤 S4，设计深度强化学习模型的训练机制

在本申请实例中，基于近端策略优化 PPO (Proximal Policy Optimization) 算法设计深度强化学习训练机制，包括构建策略评估网络(Actor-Critic Network)、时序差分学习、动作采样、状态转移与策略更新等环节，以实现策略的最优学习。

具体地，构建动作网络（Actor Network），采取多层感知机结构构建策略网络，以步骤 S2 所得状态向量为输入，将状态向量中不可行动作特征置零以过滤非法动作，输出归一化后动作空间中合法调度动作的 actor probability，用于后续动作采样。

具体地，构建价值网络 (Critic Network)，以对调度环境中的状态进行价值评估。采取包括四层全连接层的多层感知机结构，选取 ReLU 函数为激活函数，确定输出层为一维标量。并用于后续计算时序差分误差。以步骤 S2 所得状态向量为输入，输出状态值函数。

具体地，设计奖励函数，用于计算最优动作并评估策略的有效性。奖励函数同时考虑最大完工时间与机器重配置时间的总和，即 $r(S_t, S_{t+1}, a_t) = T(S_t) - T(S_{t+1}) - \text{trantime}(a_t)$ ，其中 $T(S)$ 是当前状态 S 的时刻，而 trantime 是可用重配置机器的重配置时间。从而在实现最有动态重构调度时间的同时，指导智能体学习最短动态重配置时间的调度策略。

具体地，设计动作采样机制，用于更新动作概率分布。其由策略网络输出的 actor probability 与知识引导模块输出的引导概率共同决定。使用 `torch.distributions.Categorical` 工具根据归一化后的动作概率分布进行动作采样；结合掩码过滤后的策略概率与引导概率融合为最终动作概率分布。

具体地，设计策略更新机制，基于 PPO 方法实现策略高效优化。

1. 计算 Advantage 与目标价值：

使用时序差分误差： $\delta_t = r_t + \gamma V_\phi(S_{t+1}) - V_\phi(S_t)$ ；采用 GAE（Generalized Advantage Estimation） $A_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$ 以及目标值 $TG_t = A_t + V_\phi(S_t)$ 计算 Advantage 值，提升样本利用效率；归一化 Advantage，稳定训练过程。

2. 构建损失函数并执行优化：

计算 PPO 策略损失 $\Delta_{clip} = E_t[\min(r_t A_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_t)]$ ，并采用均方误差 MSE 计算价值函数损失 $\Delta_{value} = E[(V_\phi(S_t) - TG_t)^2]$ ；采用梯度裁剪控制梯度爆炸问题以提高训练稳定性。

3. 使用 Adam 优化器分别更新策略网络与价值网络，同时更新参数 $\omega = \omega + \sum_t \delta_t \Delta_{clip} \log \pi_\omega(a_t | S_t)$ 和 $\phi = \phi + \sum_t \delta_t \Delta_{value} V_\phi(S_t)$ ，用 epoch 参数控制每轮更新重复，增强模型对样本的充分利用。

具体地，在状态转移机制和训练流程方面，在本申请实例中，采用基于环境交互的训练流程，模拟调度系统的状态迁移与策略优化过程。包括创建新实例以初始化调度环境，提取当前状态向量，进行动作采样并更新环境状态，将状态、动作、奖励等信息存入经验回放池 `ReplayBuffer` 以实现数据存储，当样本数量满足更新条件时调用 `update()` 进行训练以实现策略更新。

步骤 S4，构建云边协同决策架构

步骤 S5，构建动态调度执行系统（KGDRL 算法实施）

构建动态调度执行系统，旨在实现基于知识引导的深度强化学习算法，基于步骤 S3 构建的策略网络与知识引导模块，搭建一个完整的动态调度执行系统，执行强化学习调度策略，完成任务调度、状态更新、策略优化等完整闭环操作。系统实现以多实例并行训练为基础，通过高效训练流程实现策略模型的自主学习与迭代优化。包括模型初始化、调度环境实例并行初始化、动作采样与执行、状态更新与策略学习、训练流程控制与返回结果五部分。

具体地，在模型初始化部分，系统在开始执行前，初始化策略网络、价值网络、经验回放池、学习率、折扣因子等模型参数，为后续任务执行做好准备。

具体地，在多实例调度任务并行初始化部分，系统支持初始化多个独立调度任务实例。每个任务通过调用 `JSRS.make()` 函数创建，并通过 `env.reset()` 初始化状态，以提升训练效率与策略泛化能力。

具体地，在动作采样与执行部分，系统在每个调度周期内根据当前状态向量计算调度动作。基于所有机器与作业节点构造状态向量，并构造合法动作掩码；计算策略网络合法与非法动作概率和知识引导概率。以上述概率为基础计算最终动作概率，并采样执行动作。

具体地，在状态迁移与策略优化部分，以步骤 S4 为基础，构造奖励函数，进行状态存储，并进行策略更新；使用 `Generalized Advantage Estimation (GAE)` 计算 `Advantage`；计算 PPO 损失，其中 Actor 网络使用 `clip` 策略更新；Critic 网络使用 TD 目标拟合值函数更新。

具体地，在系统训练流程控制与输出部分，设定迭代轮数，在每轮迭代中完成多实例并行训练；每轮统计回报、最小完工时间、损失变化等指标；最终可视

化训练结果输出，包括：回报曲线、完工时间、Actor/Critic 损失等。

算法 1 : KGDRL

初始化策略网络 π_ω 和评估网络 V_ϕ ，设置可训练参数 ω 和 ϕ

初始化 β 个独立的车间调度算例

当 $iter = 1, 2, \dots, I$ 时运行:

 当 $b = 1, 2, \dots, \beta$ 时运行:

 基于 b ，初始化 S_t

 当 S_t 没有终止时运行:

 基于 S_t ，初始化 a_t ，掩码向量 F_{avail} 和 $F_{nonavail}$

$F_{avail} = 1, F_{nonavail} = 0$ if $O_{ij} \in a_t$

 基于策略网络计算动作分布概率

$$p_{avail} = \pi_\omega(F_{avail}(O_{ij})S_t), p_{nonavail} = \pi_\omega(F_{nonavail}(O_{ij})S_t)$$

 计算知识引导动作概率分布 $p_{advice} = p_{a_t}$

 基于联合动作概率 $p = p_{advice} + p_{avail} + p_{nonavail}$ 抽样最终决策动作 a

 计算奖励 r_t 和下一状态向量 S_{t+1}

$$S_t \leftarrow S_{t+1}$$

 对于每一步，基于 r_t 和 S_{t+1} 计算广义优势估计值(GAE) A_t

 基于 A_t 计算 PPO 损失 Δ

 基于 PPO 损失 Δ 更新网络参数 ω 和 ϕ

Return
